

Belirsizliğin Korunumu: Kodun Determinizminden Ajan Kısıtlarına

Ömer Faruk Yavuz

August 2026

Özet

Sıradan yazılım kodu deterministiktir; pratikte gözlenen belirsizlik kodun kendisinden değil, sabitlenmemiş gizli girdilerinden gelir. Bu girdiler sabitlendiğinde belirsizlik tümüyle kaybolur, dolayısıyla olasılık burada sistemin bir özelliği değil gözlemcinin bilgi eksiğinin ölçüsüdür. Mühendislik pratiğinde bu ayrımın sonucu, belirsizliğin ortadan kaldırılmasının değil yerinin değiştirilmesinin hedeflenmesidir: yeniden deneme mekanizması ağ belirsizliğini başarısızlıktan gecikmeye çevirir, belirsizliği yok etmez. Robert C. Martin’in 2026’da savunduğu “ajan kodunu okumama” pozisyonu bu ilkenin bir uygulamasıdır; güvenilir artefakt kodun kendisinden, kodu çevreleyen deterministik kısıt katmanına taşınmıştır. Ancak büyük dil modellerinin ürettiği belirsizlik de ontik değil epistemiktir — sıcaklık ve tohum sabitlendiğinde örnekleme deterministikleşir — ve bu, “kod yalnızca bir örneklemdir, dolayısıyla okumak anlamsızdır” argümanının dayanağını zayıflatır. Grady Booch’un itirazı ise mekanizmanın kapsama haritasındaki asıl boşluğa işaret eder: test paketi yalnızca düşünülmüş başarısızlıkları yakalar, kod okuma ise düşünülmemiş olanları yakalayabilen tek mekanizmadır. Kapsama haritasının kendi içindeki çelişki de buradadır: gereksinim hatası sınıfı için tek güvence mekanizması insan gözü olarak tanımlanırken, aynı çerçeveye insanı devreden çıkarmayı önermektedir.

Giriş

Bir hesaplama sisteminin deterministik mi yoksa olasılıksal mı olduğu sorusu, ilk bakışta teknik bir sınıflandırma sorusu gibi görünür. Pratikte ise bu soru, o sistem hakkında hangi güvence yöntemlerinin anlamlı olduğunu belirler. Deterministik bir sistemde tek bir doğrulama, sonsuz sayıda çalıştırma için geçerlidir; olasılıksal bir sistemde tek bir doğrulama yalnızca bir örnekleme kapsar. Doğrulama emeğinin nereye harcanacağı bu ayrıma bağlıdır.

Determinizm, bir sistemin aynı başlangıç durumu ve aynı girdiyle her zaman aynı çıktıyı üretmesidir. **Olasılıksallık** ise çıktının bir olasılık dağılımından çekilmesidir; aynı girdi farklı çıktılar üretebilir ve doğru davranış ancak dağılım düzeyinde tanımlanabilir.

Bu iki kategori arasında sıklıkla karıştırılan bir üçüncü durum vardır: deterministik olduğu hâlde gözlemciye olasılıksal görünen sistemler. Zar, hava durumu, sözde-rastgele sayı üreteçleri ve çok çekirdekli bir makinede çalışan eşzamanlı program bu üçüncü kümededir. Karışıklığın kaynağı, olasılığın nerede bulunduğu belirtilmemesidir: sistemde mi, yoksa sistemi betimleyen kişinin bilgisinde mi.

Yaygın ve yanlış olan kabul, mühendislik hedefinin belirsizliği sıfırlamak olduğudur. Bu kabul, kaldırılabilir belirsizlikle sistemin doğasındaki belirsizliği aynı kefiye koyar ve ikincisini zorla determinize etmeye çalışan kırılğan tasarımlar üretir.

Bu notun kapsamı, determinizm-olasılık ayrımının kuruluşu, bu ayrımın epistemik ve ontik olasılık ayrımıyla ilişkisi, buradan türeyen mühendislik ilkesi ve ilkenin büyük dil modeli ajanlarıyla çalışan yazılım geliştirme pratiğine uygulanmasıdır. Kuantum mekaniğinin yorum tartışması yalnızca ontik olasılığın varlığı sorusunun açık bırakıldığını göstermek için anılır, tartışılmaz.

Kodun Determinizmi ve Gizli Girdiler

Temel iddia

Sıradan bir program deterministiktir. `if x > 5` ifadesi belirli bir olasılıkla dallanmaz; verilen `x` değeri için ya dallanır ya dallanmaz. Bu, işlemci düzeyinde bir garantidir: makine kodu sonlu bir durum makinesidir ve her komut tanımlı bir durum geçişi üretir. Determinizm, hesaplamanın soyut modelinin — Turing makinesinin — kurucu özelliğidir (Turing, 1936).

Belirsizlik kaynakları

Pratikte gözlenen tekrarlanamazlık kodun kendisinden değil, çalıştığı ortamdan gelir. Dört ana kaynak ayırt edilebilir.

Sözde-rastgelelik

Sözde-rastgele sayı üretici (*pseudo-random number generator*, PRNG), tamamen deterministik bir fonksiyondur. Bir başlangıç değerinden — *tohum* (*seed*) — başlar ve her çağrıda iç durumunu belirli bir kuralla ilerletir. Aynı tohum her zaman aynı diziye üretir. `rand()` çağrısının rastgele görünmesi, çıktısının istatistiksel testlerden geçmesindendir, üretim sürecinin belirsiz olmasından değil.

Eşzamanlılık

İki iş parçacığının hangi sırayla ilerleyeceği program tarafından değil, işletim sistemi zamanlayıcısı tarafından belirlenir. Bu sıralama işlemci yükü, kesme zamanlaması ve önbellek durumu gibi programın göremediği değişkenlere bağlıdır. **Yarış koşulu** (*race condition*) hatalarının sinsisi olmasının sebebi tam olarak budur: kod deterministiktir, zamanlama değildir; dolayısıyla hata her çalıştırmada görünmez.

Dış durum

Sistem saati, ağ gecikmesi, dosya sisteminin içeriği ve kullanıcı girdisi programın girdisidir ve çalıştırmadan çalıştırmaya değişir. Bunlar belirsizlik kaynağı değil, değişken girdilerdir; ayırım önemlidir çünkü girdiyi sabitlemek mümkündür.

Donanım ve derleyici katmanı

Kayan nokta işlemlerinin farklı mimarilerde farklı yuvarlanması, adres uzayı düzeni rastgeleleştirmesinin (*address space layout randomization*, ASLR) bellek adreslerini değiştirmesi ve Go dilinde `map` yapısı üzerindeki döngü sırasının kasıtlı olarak karıştırılması bu katmandadır. Go'daki karıştırma, geliştiricilerin sıra üzerine yanlışlıkla bağımlılık kurmasını engellemek için dile bilinçli olarak eklenmiştir; yani belirsizliğin kendisi bir tasarım aracı olarak kullanılmıştır.

Ayrımın netleşmesi

Sayılan dört kaynağın ortak özelliği, hepsinin **gizli girdi** olmasıdır. Tohum sabitlendiğinde, yürütme tek iş parçacığına indirildiğinde ve saat taklit nesneye (*mock*) değiştirildiğinde program tamamen tekrarlanabilir hâle gelir. Belirsizlik hesaplamanın doğasında değil, sistemin bilinmeyen girdilerindedir. Otomatik testlerin yaptığı işlem tam olarak budur: gizli girdileri açığa çıkarıp sabitlemek.

Gerçek anlamda olasılıksal hesaplama ayrı bir kategoridir. Randomize algoritmalarında — Miller-Rabin asalılık testi, Monte Carlo yöntemleri — doğruluk garantisinin kendisi bir olasılık sınırı olarak ifade edilir. Miller-Rabin testi bileşik bir sayıyı yanlışlıkla asal ilan edebilir ve bu olasılık tur sayısıyla azaltılır ama sıfırlanmaz (Rabin, 1980). Burada belirsizlik ortamdan sızmaz, algoritmanın tanımına yazılmıştır.

Epistemik ve Ontik Olasılık

İzin bulunduğu yer

Deterministik bir sistem olasılıksal iz bırakmaz; tam durumuna erişimi olmayan bir gözlemci için olasılıksal görünür. Olasılık, sistemin bir özelliği değil, betimlemedeki eksiğin ölçüsüdür.

Bu ayrım Jaynes tarafından açık biçimde formüle edilmiştir: olasılık kuramı, rastgeleliğin kuramı değil, eksik bilgi altında tutarlı akıl yürütmenin genişletilmiş mantığıdır (Jaynes, 2003). Buna göre **epistemik olasılık** bilinmeyen ölçüsüdür, **ontik olasılık** ise dünyanın kendisindeki belirsizliktir.

Üç görünüm

Kaotik hassasiyet

Lojistik harita $x_{n+1} = rx_n(1 - x_n)$ tamamen deterministiktir ve tek satırlık bir formülle ifade edilir. Buna karşın r değeri kritik eşiğin üzerindeyken başlangıç koşulundaki 10^{-15} mertebesinde bir fark, birkaç yüz iterasyon içinde makroskopik ayrışmaya dönüşür (May, 1976). Sonlu hassasiyetle çalışıldığı sürece — ki her sayısal hesaplamada öyle çalışılır — öngörülebilir tek şey dağılımdır.

Hava tahmininin yüzdeyle konuşmasının sebebi budur. Atmosfer olasılıksal olduğu için değil, başlangıç durumu yeterli çözünürlükte ölçülemediği için. Lorenz'in 1963 tarihli çalışması bu duyarlılığı deterministik bir denklem sisteminde göstermiştir (Lorenz, 1963).

İstatistiksel mekanik

Newton denklemleri deterministiktir; bir gaz kabındaki 10^{23} mertebesindeki parçacığın her biri belirli bir yörünge izler. Termodinamik ise baştan sona olasılıktır. Entropi, mikro durumun bilinmemesinden doğan bir muhasebedir. Gibbs ve Boltzmann'ın kurduğu şey tam olarak bu geçiştir: deterministik alt katman ile gözlem kaybı birleştiğinde istatistiksel üst katman ortaya çıkar. Bu, epistemik olasılığın fizikteki en gelişmiş örneğidir.

Algoritmik ayırt edilemezlik

π sayısının ondalık basamakları tümüyle belirlidir, ancak bilinen hiçbir istatistiksel test onları rastgele bir diziden ayırt edemez. Kolmogorov'un ölçütü burada keskindir: bir dizinin rastgeleliği, onu üreten en kısa programın uzunluğuyla tanımlanır (Kolmogorov, 1965). π için o program kısadır, dolayısıyla dizi rastgele *değildir*; ama programı bilmeyen bir gözlemci için ayırt edilemez.

Pratik karşılık: yan kanal saldırıları

Yan kanal saldırısı (*side-channel attack*), bir şifreleme rutininin çıktısını değil, çalışma süresi, güç tüketimi veya elektromanyetik yayılım gibi yan gözlenebilirlerini

kullanarak gizli anahtarı çıkarma yöntemidir. Rutin deterministiktir; ancak çalışma süresi önbellek durumuna, dallanma tahmin biriminin durumuna ve bellek düzenine bağlı olarak dağılım gösterir. Saldırgan tek ölçümden bilgi çıkaramaz, binlerce ölçümün istatistiğinden anahtarı türetir (Kocher, 1996).

Determinizm burada bozulmamıştır. Gözlem kanalı dar olduğu için bilgi ancak dağılım biçiminde sızar. Savunma yöntemi de bu tanımla uyumludur: sabit zamanlı (*constant-time*) uygulama, çalışma süresini gizli veriden bağımsız kılarak kanalı kapatır.

Ontik olasılığın durumu

Ontik olasılığın gerçekten var olup olmadığı açık bir sorudur. Kuantum mekaniğinin standart — Kopenhag — yorumu var der; Bohm’un pilot dalga yorumu yok der ve gözlenen dağılımı gizli değişkenlere yükler (Bohm, 1952). Bu tartışma bu notun konusu değildir; buraya yalnızca “ontik olasılık” kategorisinin boş bir kategori olmadığını ama tartışmalı olduğunu belirtmek için alınmıştır.

Mühendislik açısından sonuç şudur: “deterministik mi olasılıksal mı” sorusu çoğu zaman ontolojik değil, **model seçimi** sorusudur. Durum ölçülebiliyorsa deterministik model, ölçülemiyorsa dağılım modeli kullanılır. Aynı sistem, gözlem çözünürlüğüne göre iki biçimde de modellenabilir.

Belirsizliğin Yerinin Değiştirilmesi

İki tür belirsizlik

Mühendislik pratiğinde iki farklı iş sıklıkla birbirine karıştırılır.

Kaldırılabilir belirsizlik, gizli girdilerin sabitlenmemesinden doğar. Testlerde saatin, tohumun ve yürütme sırasının sabitlenmemesi; yarış koşulunun senkronizasyonla kapatılmaması bu türdendir. Bunlar bilgi eksikliğinden değil disiplin eksikliğinden gelir ve kaldırılmalıdır; maliyeti düşük, kazancı yüksektir.

Sistemin doğasındaki belirsizlik, kullanıcının ne yazacağı, ağın ne zaman kopacağı, talebin gelecek ay ne olacağı gibi sistemin dışındaki değişkenlerden gelir. Bunları determinize etmenin yolu yoktur. Zorlandığında iki sonuçtan biri doğar: model yanlışlanır — sabit varsayılan değer gerçekte değişir — ya da sistem kırılanlaşır, çünkü tek bir senaryoya göre optimize edilmiştir ve o senaryo gerçekleşmez.

Korunum ilkesi

Belirsizlik ortadan kaldırılamaz, yeri değiştirilir. İyi tasarım belirsizliği sıfırlamak değil, onu sisteme en az zarar verecek biçime dönüştürmektir.

Mekanizma	Girdi belirsizliği	Çıktı belirsizliği
Yeniden deneme (<i>retry</i>)	Başarısız olabilir	Gecikebilir
Tampon (<i>buffer</i>)	Yük dalgalanır	Gecikme artar
Sigorta	Tekil felaket	Dağıtılmış maliyet
Devre kesici	Kademeli çökme	Kısmi hizmet kaybı

Her satırda belirsizlik korunur; yalnızca daha ucuz bir para birimine çevrilir. Yeniden deneme ağı kopmayacağını garanti etmez, kopmanın maliyetini kullanıcıya görünen bir hatadan görünmeyen bir gecikmeye taşır.

Bilerek eklenen olasılık

Rastgelelik her zaman istenmeyen bir özellik değildir; sıklıkla araçtır.

- Karma tablolarında (*hash table*) tohum rastgeleleştirilmesi, saldırganın çakışma üreterek arama karmaşıklığını $O(1)$ 'den $O(n)$ 'e çıkarmasını engeller.
- Yük dengeleyicide rastgele seçim, sunucular arasında koordinasyon gerektirmeden yaklaşık eşit dağılım sağlar; koordinasyon maliyetini sıfırlar.
- Monte Carlo yöntemleri, boyut sayısı arttıkça deterministik sayısal integrasyonun uygulanamaz hâle geldiği problemleri çözer.
- Kriptografide rastgelelik zorunludur; anahtar üretimi öngörülebilir olduğunda sistemin tamamı çöker.

Karar ölçütü

Belirsizliğin nereye yerleştirileceği sorusu şu soruyla operasyonelleştirilir: bu şeyin tekrarlanamaz olması neye mal olur. Testte tekrarlanamazlık felakettir, çünkü başarısız olan bir testi yeniden üretemeyen geliştirici hatayı bulamaz. Üretim ortamında tekrarlanamazlık normaldir, çünkü gerçek dünya zaten tekrarlanmaz. Aynı kod, iki bağlamda iki farklı politika gerektirir.

Ajan Çağında Deterministik Sınırın Kayması

Tartışmanın olgusal çerçevesi

Robert C. Martin, 14 Nisan 2026 tarihli bir X gönderisinde ajanların yazdığı kodu incelemeyi, bunun yerine test kapsamı, bağımlılık yapısı, çevrimsel karmaşıklık, modül boyutu ve mutasyon testi gibi ölçütlere baktığını yazmıştır (Martin, 2026a). 23 Temmuz 2026 tarihli ikinci gönderide pozisyonunu genişletmiş, ajanları “aşırı kısıtlarla” — birim testleri, Gherkin testleri, kalite güvence yordamları, kalite ölçütleri, mutasyon testi ve test kapsamı — kuşattığını ve bu engel dizisinden geçen koda yüksek güven duyduğunu belirtmiştir (Martin, 2026b). Bu pozisyonun arkasında yapılandırılmış bir metodoloji bulunmaktadır; O’Reilly bünyesinde “AI Agents for Clean Code” başlıklı bir canlı etkinlik olarak öğretilmektedir (explainx.ai, 2026).

Grady Booch aynı tartışmada karşı pozisyonu almış, ajanların ürettiği kodun tamamını incelediğini yazmıştır. İtirazı belirlidir: ölçütler işlevsellik konusunda güven verir, ancak ajanın bir güvenlik açığı ekleyip ekmediği, gelecekteki anlaşılabilirliği aşındıracak ölü kod bırakıp bırakmadığı ve başarımlar açısından önemli bir çarpanlaştırmayı kaçırıp kaçırmadığı konusunda hiçbir güven vermez. Booch bu itirazı “güven ama doğrula” formülüyle kapatmıştır (Booch, 2026).

Kırılan varsayım

Klasik programlamanın epistemolojik varsayımı, kodun spesifikasyonun kendisi olmasıdır. Makine tam olarak yazılanı yapar ve iki kez çalıştırıldığında aynı şeyi yapar. Bu nedenle kodu okumak anlamlıdır: okunan şey ile çalışacak şey birebir aynıdır. *Clean Code*’un tüm temeli budur; kod, insanın okuyup doğruladığı asıl artefaktır (Martin, 2008).

Ajan üretimi bu varsayımı bozar. Ajan olasılıksal bir üretectir; aynı istem aynı modele verildiğinde farklı çıktı üretebilir. Bu durumda okunan kod, sistemin ne yaptığının kalıcı ifadesi değil, bir örneklemdir. Aynı özellik yeniden üretildiğinde başka bir kod çıkacaktır. Satır satır inceleme, tek bir örneği doğrulamaya harcanan emek hâline gelir ve gelecekteki üretimler için garanti sağlamaz.

Sınırın yer değiştirmesi

Sonuç, deterministik sınırın yer değiştirmesidir. Güvenilir artefakt kod olmaktan çıkar, kodu çevreleyen kısıt katmanı olur: kabul testleri, birim testleri, Gherkin senaryoları, karmaşıklık eşikleri, modül boyutu limitleri. Bu katmanın her bileşeni deterministiktir — aynı test iki kez çalıştırıldığında aynı sonucu verir. Olasılıksal bir üretimin çevresine deterministik bir kabuk örülür ve doğrulama sorumluluğu insanın gözünden makinenin ölçütüne geçer.

Önceki ilkeyle bağı

Bu hamle, belirsizliğin korunum ilkesinin bir uygulamasıdır ve bu bağ genellikle kurulmadan bırakılır. Yeniden deneme mekanizması ağ belirsizliğini “başarısız olabilir”den “gecikebilir”e taşır; kısıt kabuğu da ajan belirsizliğini bir sorudan başka bir soruya taşır:

“Bu kod doğru mu” sorusu, “bu kısıt seti yeterli mi” sorusuna dönüşür.

Belirsizlik yok edilmemiş, taşınmıştır. Booch’un itirazının en keskin okunuşu tam buradadır: taşınan yerdeki belirsizlik daha ucuz değildir, yalnızca daha az görünürdür. Kod incelemesinde belirsizlik her satırda gözle karşılaşır; kısıt setinde belirsizlik, setin tasarlandığı gün alınan ve bir daha sorgulanmayan bir karara gömülür.

Ajan belirsizliği epistemik midir

Pozisyonun dayanağını zayıflatan bir gözlem, tartışmanın her iki tarafında da dile getirilmemiştir. Bir dil modelinin çıktısını olasılıksal kılan şey, token örnekleme aşamasıdır. Model, sözlükteki her token için bir olasılık dağılımı üretir; örnekleyici bu dağılımdan bir seçim yapar. Sıcaklık (*temperature*) parametresi sıfırlandığında ve rastgelelik tohumu sabitlendiğinde bu seçim en yüksek olasılıklı tokene indirgenir ve örnekleme deterministikleşir.

Buradan çıkan sonuç, ajan belirsizliğinin — en azından örnekleme düzeyinde — ontik değil epistemik olduğudur. Deterministik bir modelin belirlenmemiş bir girdisidir; tıpkı sabitlenmemiş bir PRNG tohumu gibi. Eğer belirsizlik kaldırılabilir sınıftansa, “okunan kod yalnızca bir örneklemdir, dolayısıyla okumak anlamsızdır” argümanının temeli kayar: sıcaklığı sıfırlanmış bir ajan, aynı istem ve aynı bağlam için aynı kodu üretir ve okunan şey yeniden kalıcı bir artefakt hâline gelir.

Bu itirazın sınırı da belirtilmelidir. Uygulamada pratik bir engel vardır: dağıtık çıkarım altyapılarında kayan nokta toplama sırasının parti (*batch*) bileşimine göre değişmesi, sıcaklık sıfır olsa bile bit düzeyinde tekrarlanabilirliği bozabilir. Ayrıca ajan oturumu yalnızca modelden ibaret değildir; araç çağrıları, alınan bağlam ve dosya sistemi durumu da girdinin parçasıdır ve bunların hepsini sabitlemek pratikte zordur. Ancak bu, belirsizliğin kategorisini değiştirmez, yalnızca sabitleme maliyetini yükseltir. Kategori epistemik kalır.

Güvence Mekanizmalarının Mekanığı

Mutasyon testi

Mutasyon testi (*mutation testing*), test paketinin gerçekten bir şey doğrulayıp doğrulamadığını ölçen yöntemdir. Araç, kaynak koda kasıtlı küçük bozmalar — *mutant* — enjekte eder:

```
if x > limit      ->   if x >= limit
return a + b      ->   return a - b
return nil        ->   return errors.New("")
```

Her mutant için test paketi çalıştırılır. Test başarısız olursa mutant “öldürülmüş” sayılır; testler o davranışı gerçekten kontrol ediyordu. Test geçmeye devam ederse mutant hayatta kalmıştır: ilgili satır çalışmaktadır ama hiçbir doğrulama ifadesi (*assertion*) sonucuna bakmamaktadır. Ölçüt şudur:

$$Mutasyonskoru = \frac{\text{öldürülenmutantsayısı}}{\text{toplammutantsayısı}}$$

Çözdüğü problem, kapsam ölçütünün yanıltıcılığıdır. Yüzde yüz satır kapsamı, hiçbir doğrulama ifadesi olmadan da elde edilebilir: fonksiyon çağrılır, satırlar kapsanmış görünür, davranış doğrulanmamıştır. Kapsam kodun çalıştığını ölçer, mutasyon testi testin baktığını ölçer. Kod okunmadığında tek kanıt test paketi olduğu için, testin kendisini denetleyen bir katman zorunlu hâle gelir.

Yöntem 1970’lerin sonunda önerilmiştir (DeMillo, Lipton ve Sayward, 1978). Maliyeti yüksektir: N mutant için test paketi N kez çalışır. Araçlar dile göre değişir — Go’da `gremlins` ve `go-mutesting`, Java’da `PIT`, JavaScript ve .NET’te `Stryker`, Python’da `mutmut`.

Çevrimsel karmaşıklık

Çevrimsel karmaşıklık (*cyclomatic complexity*), bir fonksiyonun kontrol akış grafiğindeki lineer bağımsız yol sayısıdır. McCabe tarafından 1976’da tanımlanmıştır (McCabe, 1976). Pratikte karar noktası sayımıyla hesaplanır:

$$CC = 1 + (\text{if} + \text{for} + \text{case} + \&\& + || + \text{catchsayısı})$$

Düz akan bir fonksiyonun değeri 1’dir. Dört koşul, bir döngü ve iki mantıksal bağlaç içeren bir fonksiyonun değeri 8 olur. Yaygın eşikler 10 üzeri uyarı, 15 üzeri ret biçimindedir; bu eşikler ampirik bir türetmeden çok yerleşik pratiktir.

Çözdüğü problem şudur: ajan bir özelliği, mevcut fonksiyona koşul ekleyerek de gerçekleştirebilir. Testler geçer, davranış doğrudur, ancak fonksiyon iki yüz satır ve otuz dallı hâle gelmiştir. Testler bunu görmez, çünkü testler davranışa bakar, yapıya bakmaz. Karmaşıklık eşiği yapısal bozulmanın mekanik vekilidir.

Ölçütün ikinci işlevi test edilebilirlikle doğrudan ilişkisidir ve genellikle atlanır: çevrimsel karmaşıklığı 30 olan bir fonksiyonun tüm bağımsız yollarını kapsamak için en az 30 test yolu gerekir. Yüksek karmaşıklık değeri, tam dal

kapsamının pratikte sağlanmadığının dolaylı kanıtıdır. McCabe metriği zaten bu amaçla, test sayısı için alt sınır üretmek üzere önermiştir.

Bir varyant olarak **bilişsel karmaşıklık** (*cognitive complexity*) iç içe geçmeyi ağırlıklandırır: düz bir **switch** yapısını cezalandırmaz ama üç kat iç içe geçmiş koşulları sert biçimde cezalandırır (SonarSource, 2018). Yapısal bozulmanın daha iyi bir vekilidir. Go’da **gocyclo** ve **gocognit** bu iki ölçütü hesaplar.

Bağımlılık analizi

Bağımlılık analizi, modül grafiğinin yönünü ve şeklini ölçer; hangi modülün hangisini içe aktardığını inceler. Martin’in kendi ölçüt kümesi buradan gelir (Martin, 2002):

- *Afferent coupling* (C_a): bu pakete bağımlı modül sayısı.
- *Efferent coupling* (C_e): bu paketin bağımlı olduğu modül sayısı.
- Kararsızlık: $I = C_e / (C_a + C_e)$. Sıfır tamamen kararlı, bir tamamen kararsızdır.
- Soyutluk (A): pakette soyut tiplerin oranı.
- Ana diziden uzaklık: $D = |A + I - 1|$. Soyut ama kimsenin kullanmadığı paketler ile somut ama herkesin bağlı olduğu paketler bu çizgiden sapar.

İki kural bu ölçütlerin üzerine kurulur. *Acyclic Dependencies Principle* paket grafiğinde döngü bulunmamasını gerektirir. *Dependency Rule* kaynak bağımlılıklarının içe, politikaya doğru akmasını gerektirir; alan (*domain*) katmanını **database/sql**, **net/http** veya bir nesne depolama istemcisini içe aktarmaz.

Çözdüğü problem için en kritikidir: **mimari erozyon testlerle hiç görünmez**. Alan katmanı doğrudan bir veritabanı sürücüsünü içe aktarsa, test paketi harfi harfine aynı sonucu verir. Ajan bu ihlali doğal olarak üretir, çünkü en kısa yol her zaman katmanlı delip geçmektir. Bu nedenle mimari kural insan gözü yerine sürekli tümleştirme hattında mekanik bir kapıya dönüştürülür: Go’da **go-arch-lint**, **depguard**, **import-boss**; Java’da **ArchUnit**; .NET’te **NetArchTest**; JavaScript’te **dependency-cruiser**.

Bölüşüm

Katman	Yakaladığı
Mutasyon testi	Testler sahte mi
Çevrimsel karmaşıklık	Lokal yapı bozuldu mu
Bağımlılık analizi	Mimari aşındı mı

Üçünün de yakalamadığı boşluk, kimsenin aklına gelmediği için test yazılmamış davranışlardır. Mutasyon testi var olan testleri denetler; olmayan testi üretemez.

Başarısızlık Sınıfları ve Kapsama Haritası

Kavramın işlevi

Başarısızlık sınıfı (*failure class*), bir yazılımın bozulabileceği tipleri kategorilere ayırmaktır; tekil hatalar değil, hataların ait olduğu aileler. Gerekliliğinin sebebi şudur: hiçbir güvence mekanizması — test, ölçüt, inceleme — her hatayı yakalamaz, yalnızca belirli bir aileye bakar. Hangi mekanizmanın hangi aileye baktığı bilinmediğinde, bütün mekanizmalar aynı aileye bakarken “her şey kapsandı” izlenimi doğar.

Sınıflar

Doğruluk

Kod yanlış şeyi hesaplar: $<$ yerine $\leq$, ters işaret, yanlış formül. Birim testi yakalar.

Bütünleşme

Parçalar tek tek doğru, birlikte yanlıştır. Bir servis milisaniye gönderirken diğeri saniye bekler; her ikisinin de kendi testi geçer. Birim testi yakalamaz, entegrasyon testi yakalar.

Eşzamanlılık

Yarış koşulu, kilitlenme, kaybolan güncelleme. Kod her çalıştığında farklı davranır. Deterministik test bu sınıfı ancak zaman zaman yakalar; en sinsi sınıflardan biridir. Yarış dedektörleri ve deterministik yeniden oynatma araçları kısmi güvence sağlar.

Kaynak

Bellek sızıntısı, kapatılmayan dosya tanıtıcısı, tükenen bağlantı havuzu. Test paketi otuz saniye çalışır ve hiçbir şey görmez; üretimde üçüncü günde ortaya çıkar. Yük testi ve profil çıkarma araçları yakalar.

Sınır ve girdi

Boş dizi, boş gösterici, negatif değer, çok baytlı karakter, sıfır uzunluklu dosya. Bu sınıfa özel yöntemler özellik tabanlı test (*property-based testing*) ve bulanıklaştırmadır (*fuzzing*); ikisi de girdi uzayını el yazımı örneklerin ötesinde tarar.

Güvenlik

Enjeksiyon, yetki atlaması, sızan sır. İşlevsel testlerin tamamı geçebilirken sistem yine de açık olabilir. Statik ve dinamik güvenlik analizi (SAST/DAST) ile tehdit modelleme yakalar.

Başarım

Sonuç doğru, süre kabul edilemez. $N+1$ sorgu, $O(n^2)$ döngü. Yalnızca ölçekte görünür; yük testi ve profil çıkarma gerektirir.

Mimari erozyon

Çalışır, doğrudur, yapı çürümüştür. Katman ihlali, döngüsel bağımlılık. Hiçbir davranış testi görmez; yalnızca bağımlılık analizi görür.

Operasyonel

Kod doğru, yapılandırma yanlıştır. Yanlış ortam değişkeni, eksik göç betiği, hatalı sır. Test ortamında yoktur, üretimde vardır.

Gereksinim

Kod spesifikasyona uygundur, spesifikasyon yanlıştır. Doğru şeyi kusursuz yapan ama yanlış şey olan yazılım. Hiçbir otomatik araç bu sınıfı yakalayamaz.

Kapsama haritası

Mekanizma	Baktığı sınıf	Kör olduğu
Birim testi	Doğruluk, sınır	Bütünleşme, kaynak, mimari, gereksinim
Entegrasyon testi	Bütünleşme, operasyonel	Eşzamanlılık, güvenlik, gereksinim
Mutasyon testi	Testlerin kendisi	Tüm üretim sınıfları; meta katmandır
Karmaşıklık ölçütü	Yapısal bozulma	Tüm davranışsal sınıflar
Bağımlılık analizi	Mimari erozyon	Tüm davranışsal sınıflar
Bulanıklaştırma	Sınır, güvenlik	Doğruluk, gereksinim
Kod okuma	Gereksinim, tasarım, sezilmemiş vakalar	Ölçek, eşzamanlılık

Tablodaki çelişki

Tartışma tam olarak son satırdır. Martin ilk altı satırın yeterli kapsama sağladığını savunur; Booch son satırın kapsadığı alanın başka hiçbir mekanizmada karşılığı olmadığını savunur.

Haritanın kendi içinde açık bir tutarsızlık barındırdığı ve bunun genellikle fark edilmediği belirtilmelidir. Gereksinim sınıfı için tek güvence mekanizması insan gözü olarak tanımlanmıştır; buna karşın kısıt kabuğu stratejisi insanı doğrulama zincirinden çıkarmayı önermektedir. İki ifade aynı anda savunulamaz. Tutarlı okuma, insanın devreden çıkmadığı ama katmanı değiştirdiğidir: insan artık kodu değil kabul ölçütlerini okur. Bu, gereksinim sınıfını gerçekten kapsar — ancak yalnızca kabul ölçütlerinin gerçekten insan tarafından yazıldığı ve okunduğu ölçüde. Ajan hem kodu hem testleri hem de kabul ölçütlerini üretiyorsa gereksinim sınıfı tümüyle kapsamsız kalır ve hiçbir ölçüt bunu bildirmez.

Goodhart problemi

İkinci yapısal itiraz ölçüt-hedef dönüşümüdür. Bir ölçüt hedefe dönüştüğünde iyi bir ölçüt olmaktan çıkar (Goodhart, 1975; Strathern, 1997). Ajanlar açıkça ölçülen kapağı karşı optimize eder: hedef kapsam olduğunda her satırı çalıştıran ama hiçbir şey doğrulamayan testler üretmeyi öğrenir. Mutasyon testi bu belirli test sınıfını öldürür; ancak mutasyon skorunun kendisi hedefe dönüştüğünde aynı mekanizma bir üst katmanda tekrarlanır. Karmaşıklık eşiği için de aynısı geçerlidir: eşiği geçen ama yapısal olarak kötü kod — örneğin mantığı yapay biçimde bölünmüş, birbirine sıkı bağlı küçük fonksiyonlar dizisi — üretilebilir ve ölçüt bunu iyi olarak raporlar.

Pratik kullanım

Bir hata üretime kaçtığında sorulacak soru “bu hata nasıl düzeltilir” değil, “bu hata hangi sınıfa aittir ve o sınıfa bakan bir mekanizma var mıydı” olmalıdır. Cevap olumsuzsa sorun kodda değil güvence portföyündedir. Aynı sınıftan üçüncü hata geldiğinde tekil düzeltme yerine o sınıfı kapsayan mekanizmanın eklenmesi gerekir.

İki Epistemoloji: Martin ve Fowler

Robert C. Martin ve Martin Fowler aynı kuşaktır ve her ikisi de 2001 tarihli Agile Manifesto'nun imzacılarından. Buna karşın epistemolojileri farklıdır ve öğrenilebilecek şey tam olarak o farktır.

Martin'den

Kod tek gerçek spesifikasyondur

Tasarım dokümanı çürür, kod çürümez; çünkü çalışan tek şey odur. Bu nedenle okunabilirlik bir konfor meselesi değil, spesifikasyonun okunabilirliğidir.

Bağımlılık yönü mimarının kendisidir

Clean Architecture'ın tek cümlelik özeti, kaynak bağımlılıklarının içe, politikaya doğru akması ve detayın — veritabanı, çerçeve, arayüz — dışarıda kalmasıdır (Martin, 2017). Sağladığı asıl kabiliyet kararı erteleyebilmektir: hangi veritabanının kullanılacağı sona bırakılabiliyorsa mimari iyidir.

Disiplin yetenekten daha iyi ölçeklenir

TDD, SOLID ve izci kuralı, “iyi mühendis ol” demek yerine “iyi mühendis olmayı zorunlu kılan mekanizma kur” der. 2026 pozisyonu bunun doğal uzantısıdır: disiplin insandan sürekli tümleştirme kapılarına taşınmıştır.

Zayıf yanı

Normatiflik. Kurallar bağlamdan bağımsız, evrensel ve zaman zaman ahlaki bir dille sunulur. “Fonksiyonlar dört ila yirmi satır olmalıdır” türü eşikler ampirik değil estetik yargılardır. Ousterhout'un itirazı buradadır: aşırı parçalama, okuyucuyu on dosya arasında dolaştırarak anlamayı kolaylaştırmak yerine zorlaştırır (Ousterhout, 2018).

Fowler'dan

Refactoring bir katalogdur

Fowler'ın asıl katkısı “kodu temizle” öğüdü değil, dönüşümlerin her birine ad, ön koşul ve mekanik adım listesi vermesidir: Extract Method, Move Field, Replace Conditional with Polymorphism (Fowler, 1999). Davranış korunur, yapı değişir. Ad vermek, işlemi tartışılabilir kılar.

Kavramlara ad koymak kavramı kullanılabilir kılar

Fowler'ın yaydığı terimler: teknik borç (Cunningham'dan devralınmıştır), kod kokusu, strangler fig, feature toggle, branch by abstraction, bounded context,

CQRS, mikroservis ön koşulları. Bu terimlerin varlığı, ekiplerin ilgili problemleri tartışabilmesinin ön koşuludur; adı olmayan problem gündeme alınmaz.

Bağlam mutlakdan önce gelir

Fowler tipik olarak “şu koşullarda bunu yap, şu koşullarda tersini yap” biçiminde yazar. MonolithFirst bunun en net örneğidir: mikroservis savunucusu olmasına rağmen, sınırlar bilinmeden mikroservisle başlamanın başarısızlıkla sonuçlandığını yazmıştır (Fowler, 2015). Aynı çizgide “you must be this tall to use microservices” formülü, bir mimari tarzın ön koşulları olduğunu, tarzın kendisinin iyi ya da kötü olmadığını söyler.

Evrimsel mimari

Mimari, önden verilen kararlar kümesi değil, değiştirilebilir tutulan kararlar kümesidir. **Uygunluk fonksiyonu** (*fitness function*) kavramı buradan gelir: mimari niteliğin otomatik ölçülebilir bir teste dönüştürülmesi (Ford, Parsons ve Kua, 2017).

Zayıf yanı

Her şeyin duruma bağlı olması, karar vermek zorunda olan kişiye az yön verir. Martin’in çekiciliği tam olarak bu boşluktan doğar.

Kesişim

Değişim maliyeti tasarımın tek ölçütüdür. Her ikisi de “güzel kod” argümanı kurmaz; ikisi de gelecekteki değişimin maliyetinden söz eder. Fowler’ın Design Stamina Hypothesis’i bunu bir grafiğe döker: tasarısız ilerleme kısa vadede hızlıdır, bir eşikten sonra eğriler kesişir (Fowler, 2007).

Testler doğrulama aracı değil tasarım baskısıdır. Test edilmesi zor kod kötü tasarlanmış koddur, çünkü test yazmak kodu dışarıdan kullanmaya zorlar. Bu, ikisinin de merkezî tezidir.

Geri bildirim döngüsü kısaltılır. Sürekli tümleştirme, küçük commit ve sürekli refactoring, aynı ilkenin farklı ölçeklerdeki uygulamalarıdır: hatanın oluşumu ile keşfi arasındaki süreyi düşürmek.

Ajan bağlamına uygulanışı

Martin’in pozisyonu kendi sistemi içinde tutarlıdır: disiplin mekanizmalastırılabiliriyorsa, mekanizmayı kimin çalıştırdığı — insan ya da ajan — önemsizdir. Ajanı kısıt duvarıyla çevrelemek, TDD’yi otomatikleştirmenin devamıdır.

Fowler’ın çerçevesi farklı bir soru sordurur: bu pratiğin ön koşulları nelerdir. MonolithFirst mantığıyla okunduğunda strateji, olgun bir test paketi, net mimari kurallar ve iyi tanımlanmış bir alan zaten mevcutsa çalışır. Keşif aşamasındaki, spesifikasyonu belirsiz bir projede aynı strateji, yanlış şeyin kusursuz

test edilmiş biçimde inşa edilmesine yarar. Uygunluk fonksiyonu kavramı Martin'in ölçüt kapılarının doğrudan karşılığıdır; farkı, Fowler'da bunların mutlak eşikler değil projeye özgü ve zamanla gözden geçirilen ölçütler olmasıdır.

Pratik çıkarım, mekanizmanın Martin'den, mekanizmanın geçerlilik koşulunun Fowler'dan alınmasıdır.

Sonuç

Determinizm-olasılık ayrımının kodda kurulan hâli, güvence yöntemlerinin seçimini doğrudan belirler. Kod deterministik olduğu için tek bir doğrulama sonsuz çalıştırma için geçerlidir; bu, kod incelemesinin klasik gerekçesidir. Ajan üretimi bu gerekçeyi zayıflatır, ancak kaldırmaz — çünkü ajan belirsizliği de epistemik sınıftandır ve sabitlenebilir. Tartışmanın gerçek eksenini “kod okunmalı mı” değil, “sabitleme maliyeti, okuma maliyetinden düşük mü” sorusudur; bu soru proje ölçeğine ve altyapıya göre farklı cevaplanır.

Belirsizliğin korunum ilkesi, güvence mimarisinin değerlendirilmesinde tek ölçüt olarak kullanılabilir. Kısıt kabuğu belirsizliği yok etmez, “bu kod doğru mu” sorusundan “bu kısıt seti yeterli mi” sorusuna taşır. Bu taşımının kârlı olup olmadığı, ikinci sorunun ilkinden daha seyrek ve daha ucuz sorulup sorulmadığına bağlıdır. Olgun bir alanda ikinci soru yılda birkaç kez sorulur ve kâr açıktır; keşif aşamasındaki bir projede her özellikle birlikte yeniden sorulması gerekir ve kâr kaybolur.

Kapsama haritasının pratik değeri, mekanizma listesinin uzunluğunun kapsama garantisi olmadığını göstermesidir. Yedi mekanizmanın altısı davranışa bakıyorsa, mimari erozyon ve gereksinim hataları için kapsam sıfırdır. Portföy değerlendirmesi mekanizma sayısıyla değil, örtülen sınıf sayısıyla yapılır.

Sınırlılıklar

Epistemik-ontik ayrımı mühendislik pratiğinde her zaman keskin değildir. Bir belirsizliğin “kaldırılabilir” sayılması, kaldırma maliyetinin kabul edilebilir olmasına bağlıdır; maliyet yeterince yükseldiğinde epistemik belirsizlik pratikte ontik gibi davranır. Bu notta çizilen ayrım, kategorik olmaktan çok maliyet eşliğine göreli bir ayrımdır.

Başarısızlık sınıfları listesi tüketici değildir ve sınıflar birbirini dışlamaz. Bir bellek sızıntısı hem kaynak hem başarımlar sınıfına girebilir; bir yapılandırma hatası güvenlik açığı üretebilir. Sınıflandırmanın işlevi kesin bölme yapmak değil, kör nokta aramayı sistematikleştirmektir.

Martin ve Booch tartışmasının olgusal zemini 2026 yılına ait ve gelişmekte olan bir tartışmadır; her iki pozisyonun da bu notta aktarılandan farklı biçimde revize edilmiş olması mümkündür.

Açık sorular

Sıcaklığı sıfırlanmış ve tohumu sabitlenmiş bir ajanın gerçekten tekrarlanabilir kod ürettiği, dağıtık çıkarım altyapıları üzerinde deneysel olarak doğrulanmalıdır. Bu doğrulama, notta kurulan epistemik sınıflandırma argümanının ampirik dayanağıdır ve burada varsayım olarak bırakılmıştır.

Kabul ölçütlerinin kendisinin ajan tarafından üretildiği durumda gereksinim sınıfını kapsayacak bir mekanizmanın var olup olmadığı açıktır. Mutasyon testi testleri denetler; kabul ölçütlerini denetleyecek bir üst katmanın ne olacağı tanımlanmamıştır.

Kaynaklar

- Bohm, D. (1952). A Suggested Interpretation of the Quantum Theory in Terms of “Hidden” Variables. *Physical Review*, 85(2).
- Booch, G. (2026). X gönderisi, Martin’in 14 Nisan 2026 tarihli gönderisine yanıt. Aktaran: Startup Fortune, “Uncle Bob Martin says he no longer reads AI-generated code and the developer world is split”.
- DeMillo, R. A., Lipton, R. J. ve Sayward, F. G. (1978). Hints on Test Data Selection: Help for the Practicing Programmer. *IEEE Computer*, 11(4).
- explainx.ai (2026). Uncle Bob: Skip Code Review, Build a Test Gauntlet.
- Ford, N., Parsons, R. ve Kua, P. (2017). *Building Evolutionary Architectures*. O’Reilly.
- Fowler, M. (1999). *Refactoring: Improving the Design of Existing Code*. Addison-Wesley.
- Fowler, M. (2007). Design Stamina Hypothesis. *martinfowler.com*.
- Fowler, M. (2015). MonolithFirst. *martinfowler.com*.
- Goodhart, C. (1975). Problems of Monetary Management: The U.K. Experience. Ayrıca bkz. Strathern, M. (1997). “Improving Ratings”: Audit in the British University System. *European Review*, 5(3).
- Jaynes, E. T. (2003). *Probability Theory: The Logic of Science*. Cambridge University Press.
- Kocher, P. (1996). Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems. *CRYPTO ’96*.
- Kolmogorov, A. N. (1965). Three Approaches to the Quantitative Definition of Information. *Problems of Information Transmission*, 1(1).
- Lorenz, E. N. (1963). Deterministic Nonperiodic Flow. *Journal of the Atmospheric Sciences*, 20(2).
- Martin, R. C. (2002). *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall.
- Martin, R. C. (2008). *Clean Code*. Prentice Hall.
- Martin, R. C. (2017). *Clean Architecture*. Prentice Hall.
- Martin, R. C. (2026a). X gönderisi, 14 Nisan 2026.
- Martin, R. C. (2026b). X gönderisi, 23 Temmuz 2026.

- May, R. M. (1976). Simple Mathematical Models with Very Complicated Dynamics. *Nature*, 261.
- McCabe, T. J. (1976). A Complexity Measure. *IEEE Transactions on Software Engineering*, SE-2(4).
- Ousterhout, J. (2018). *A Philosophy of Software Design*. Yaknyam Press.
- Rabin, M. O. (1980). Probabilistic Algorithm for Testing Primality. *Journal of Number Theory*, 12(1).
- SonarSource (2018). *Cognitive Complexity: A New Way of Measuring Understandability*. Teknik rapor.
- Turing, A. M. (1936). On Computable Numbers, with an Application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 2(42).